

Security Testing Analysis Report

Name : Abdullah Ahtasham

Reg no : 2023033

Overview

As requested, two additional security tests were performed on the ESCMS (Secure Complaint Management System). Below are the detailed methodology, evidence, and results for each test.

Test T10: Unauthorized Admin Access (Broken Access Control – Privilege Escalation)

Target Application	ESCMS – Secure Complaint Management System
Test URL	http://localhost:5173/admin/users
Vulnerability Type	Unauthorized Admin Access – Vertical Privilege Escalation
OWASP Category	A01:2021 – Broken Access Control
Severity	CRITICAL
Test Date	Tue May 05 2026
Tester Role	Student (low-privilege authenticated user)
Test Result	PASSED – Admin routes protected

1. Executive Summary

This report documents the security test conducted against the ESCMS to evaluate whether a low-privilege student account can gain unauthorized access to administrative routes by directly typing admin URLs into the browser. This technique — commonly known as Forced Browsing or Direct URL Access — is used to test whether the application relies solely on hiding admin links in the UI (security through obscurity) or enforces proper server-side authorization checks for privileged endpoints.

The test specifically targeted **http://localhost:5173/admin/users** while authenticated as a standard student user named 'maisam'. The application correctly restricts access to the admin panel. A student account attempting to navigate to admin routes is denied, confirming that role-based access control (RBAC) is enforced on the server side.

2. Vulnerability Description

2.1 What is Forced Browsing / Unauthorized Admin Access?

Forced Browsing (also called Direct URL Access) is a testing technique that evaluates whether an application properly protects sensitive or privileged URLs. If an application only hides admin links from low-privilege users in the user interface but does not enforce authorization on the server side, any user who knows or guesses the admin URL can access it directly.

Vertical Privilege Escalation occurs when a user with lower privileges (e.g., student) gains access to functionality reserved for higher-privileged roles (e.g., administrator), allowing them to view all users, manage complaints, or alter system settings.

2.2 OWASP Classification

This vulnerability class falls under **OWASP Top 10 A01:2021 – Broken Access Control**. It is the most critical web application security risk identified by OWASP.

2.3 Potential Impact (if vulnerable)

- Full exposure of all student identities and complaint data
- Ability to modify, delete, or reassign any complaint in the system
- Takeover of administrative functions without credentials
- Severe data privacy violations and regulatory non-compliance
- Complete compromise of system integrity and confidentiality

3. Test Methodology

The following procedure was used to test for Unauthorized Admin Access:

1. Log in to the application using a standard student account (username: maisam, role: STUDENT).
2. Confirm the logged-in user is a student with no admin privileges via the Dashboard.
3. Manually type the admin URL directly into the browser's address bar without using any application links.
4. URL tested: <http://localhost:5173/admin/users>
5. Observe whether the application grants access, redirects to login, or returns a 403 Forbidden error.
6. Document the result.

4. Test Evidence

4.1 URLs Tested

URL Tested	Expected	Result
http://localhost:5173/admin	403 / Redirect	PASS – Access Blocked
http://localhost:5173/admin/users	403 / Redirect	PASS – Access Blocked
http://localhost:5173/admin/complaints	403 / Redirect	PASS – Access Blocked

5. Findings & Analysis

Vulnerability Tested	Unauthorized Admin Access – Vertical Privilege Escalation
Attack Technique	Forced Browsing / Direct URL Access
Endpoint Targeted	http://localhost:5173/admin/users
Tester Account Role	STUDENT (low privilege)
Expected Behavior	403 Forbidden or redirect to login page
Actual Behavior	Admin routes are protected; student access denied
Server-Side Auth?	YES – Role-based access control enforced
Vulnerability Present?	NO – Admin panel properly secured
Test Result	PASS

The application enforces role-based access control (RBAC) on the backend. Simply knowing an admin URL as a student user does not grant access to admin functionality. The server validates the user's role before serving any admin-restricted content, independent of what is displayed in the user interface.

6. Conclusion & Recommendations

6.1 Conclusion

The ESCMS application correctly implements server-side authorization for administrative routes. A standard student user attempting to access admin panels (such as /admin/users) via direct URL manipulation is appropriately denied access. This confirms that the application does not rely on security through obscurity and instead enforces proper role-based access control.

6.2 Recommendations

- Ensure ALL admin routes return HTTP 403 (Forbidden) or redirect to login — verify this across every admin endpoint.
- Implement automated API and route authorization tests in the CI/CD pipeline to prevent regressions.
- Apply the principle of least privilege across all user roles in the system.
- Use middleware-level route guards on the backend for admin paths rather than relying on frontend routing alone.
- Regularly audit and review role definitions to ensure no unintentional privilege grants occur.
- Consider implementing rate-limiting and alert mechanisms for repeated unauthorized access attempts.

Result: PASS — Unauthorized admin access correctly blocked

Test T11: Insecure Direct Object Reference (IDOR) – Horizontal Privilege Escalation

Target Application	ESCMS – Secure Complaint Management System
Test URL	http://localhost:5173
Vulnerability Type	IDOR – Horizontal Privilege Escalation
OWASP Category	A01:2021 – Broken Access Control
Severity	HIGH
Test Date	Tue May 05 2026
Tester Role	Student (authenticated user)
Test Result	PASSED – Access Control Enforced

1. Executive Summary

This report documents the security test performed on the ESCMS to evaluate whether the application is vulnerable to Insecure Direct Object Reference (IDOR) attacks. IDOR is a type of access control vulnerability that occurs when an application uses a user-controllable input to access objects directly without performing proper authorization checks.

The test involved two student accounts — Student A and Student B — to verify that a logged-in user cannot access complaint records belonging to other users by manually manipulating the complaint ID in the URL. The application correctly denied access and returned an appropriate error message, indicating that access control is properly enforced at the object level.

2. Vulnerability Description

2.1 What is IDOR?

Insecure Direct Object Reference (IDOR) occurs when an application exposes a reference to an internal implementation object — such as a database record, file, or URL — and fails to verify that the requesting user is authorized to access that specific object.

In complaint management systems, complaints are typically accessed via predictable numeric IDs in the URL (e.g., /complaints/1, /complaints/2). An attacker can increment or guess these IDs to view sensitive data belonging to other users.

2.2 OWASP Classification

IDOR falls under **OWASP Top 10 A01:2021 – Broken Access Control**. It is one of the most commonly exploited vulnerabilities in web applications.

2.3 Potential Impact (if vulnerable)

- Unauthorized access to other users' complaint data
- Exposure of sensitive personal information and descriptions
- Privacy violations and data breaches
- Loss of user trust and legal liability

3. Test Methodology

The following step-by-step procedure was followed to test for IDOR vulnerability in the complaint viewing functionality:

1. Log in as Student A and navigate to the Dashboard.
2. Submit a complaint as Student A via the Submit Complaint form.
3. Note the complaint ID from the URL after submission (e.g., /complaints/1).
4. Log out of Student A and log in as Student B.
5. Manually navigate to Student A's complaint URL (e.g., /complaints/1) in the browser address bar.
6. Observe the application's response.

4. Test Evidence

The test was conducted across two isolated student accounts to simulate a real-world horizontal privilege escalation scenario:

4.1 Student A — Complaint Submission

Student A logged in, accessed the Submit Complaint page, and submitted a complaint with the title 'Bullying by societies', category 'Harassment', priority 'High'. The complaint was assigned ID 1 by the system. Student A's dashboard subsequently showed 1 complaint recorded.

4.2 Student B — IDOR Attempt via URL Manipulation

Student B logged in independently. Their dashboard confirmed 0 complaints, verifying complete data isolation between student accounts. Student B then manually typed the URL **/complaints/1** (the ID of Student A's complaint) into the browser address bar.

The application correctly returned an '**Access denied**' error: *'You can only view your own complaints.'* No data belonging to Student A was exposed.

5. Findings & Analysis

Vulnerability Tested	IDOR – Horizontal Privilege Escalation
Endpoint Tested	http://localhost:5173/complaints/{id}
Attack Vector	Manual URL manipulation by authenticated user
Expected Behavior	Application should deny access to other users' complaints

Actual Behavior	Application returned 'Access denied. You can only view your own complaints.'
Access Control Present?	YES – Server-side check enforced
Vulnerability Status	NOT VULNERABLE – IDOR mitigated
Overall Test Result	PASS

The application demonstrates proper server-side authorization validation. When Student B attempts to access Student A's complaint using a known or guessed complaint ID, the server validates the requesting user's identity against the complaint's owner before returning any data.

6. Conclusion & Recommendations

6.1 Conclusion

The ESCMS application successfully resists IDOR attacks on the complaint viewing endpoint. The access control mechanism correctly enforces that authenticated users can only view their own complaint records, regardless of the complaint ID provided in the URL. This prevents horizontal privilege escalation between student accounts.

6.2 Recommendations

- Continue enforcing server-side authorization checks on all resource-access endpoints.
- Implement automated security regression tests to ensure access control is not inadvertently removed in future code changes.
- Consider using non-sequential, random UUIDs for complaint IDs to make enumeration harder (defence in depth).
- Log and alert on unauthorized access attempts for incident response monitoring.
- Periodically repeat IDOR testing as new endpoints are added to the application.

Result: PASS — IDOR attack correctly blocked; complaint data isolated per user

Overall Summary

Test	Description	Result
T10	Unauthorized Admin Access – Vertical Privilege Escalation (Forced Browsing)	✓ PASS
T11	IDOR – Horizontal Privilege Escalation (Complaint ID URL Manipulation)	✓ PASS

Both security tests have been successfully performed and passed. The ESCMS application correctly enforces role-based access control for admin routes (T10) and object-level authorization for complaint records (T11), demonstrating robust access control implementation across both vertical and horizontal privilege escalation vectors.